

```
# =====
# Effect of Noise on Harris Corner Detection
# Google Colab Compatible
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# =====
# Upload Image
# =====

uploaded = files.upload()

image_path = list(uploaded.keys())[0]

img = cv2.imread(image_path)

if img is None:
    raise Exception("Could not load image.")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# =====
# Function to Add Gaussian Noise
# =====

def add_gaussian_noise(image, sigma):

    noise = np.random.normal(
        0,
        sigma,
        image.shape
    )

    noisy = image.astype(np.float32) + noise

    noisy = np.clip(
        noisy,
        0,
        255
    )

    return noisy.astype(np.uint8)

# =====
# Create Noisy Images
# =====

low_noise = add_gaussian_noise(gray, 5)

medium_noise = add_gaussian_noise(gray, 15)

high_noise = add_gaussian_noise(gray, 30)

# =====
# Harris Corner Detection Function
# =====

def harris_corners(gray_image):

    # Light blur to stabilize gradients
    gray_blur = cv2.GaussianBlur(
        gray_image,
        (3,3),
        0
    )

    gray_float = np.float32(gray_blur)

    dst = cv2.cornerHarris(
        gray_float,
        blockSize=2,
        ksize=3,
        k=0.04
    )

    dst = cv2.dilate(dst, None)

    # Tuned threshold
    threshold = 0.07 * dst.max()

    result = cv2.cvtColor(
        gray_image,
        cv2.COLOR_GRAY2RGB
    )

    result[dst > threshold] = [255, 0, 0]

    # Count corner regions instead of pixels
    mask = np.uint8(dst > threshold)

    num_labels, labels = cv2.connectedComponents(mask)

    corner_count = num_labels - 1

    return result, corner_count

# =====
# Detect Corners
# =====

original_corners, c1 = harris_corners(gray)

low_corners, c2 = harris_corners(low_noise)

medium_corners, c3 = harris_corners(medium_noise)

high_corners, c4 = harris_corners(high_noise)

# =====
# Improvement Technique
# Gaussian Blur on High Noise Image
# =====

improved_image = cv2.GaussianBlur(
    high_noise,
    (5,5),
    0
)

improved_corners, c5 = harris_corners(
    improved_image
)

# =====
# Display Results
# =====

plt.figure(figsize=(16,10))

plt.subplot(2,3,1)
plt.imshow(original_corners)
plt.title(f'Original\nCorners = {c1}')
plt.axis('off')

plt.subplot(2,3,2)
plt.imshow(low_corners)
plt.title(f'Low Noise\nCorners = {c2}')
plt.axis('off')

plt.subplot(2,3,3)
plt.imshow(medium_corners)
plt.title(f'Medium Noise\nCorners = {c3}')
plt.axis('off')

plt.subplot(2,3,4)
plt.imshow(high_corners)
plt.title(f'High Noise\nCorners = {c4}')
plt.axis('off')

plt.subplot(2,3,5)
plt.imshow(improved_corners)
plt.title(f'Blurred + Harris\nCorners = {c5}')
plt.axis('off')

plt.tight_layout()
plt.show()

# =====
# Result Summary
# =====

print("\n=====")
print(" HARRIS CORNER DETECTION RESULTS")
print("=====")
```

```
print(f"Original Image      : {c1}")
print(f"Low Noise           : {c2}")
print(f"Medium Noise          : {c3}")
print(f"High Noise              : {c4}")
print(f"After Gaussian Blur   : {c5}")

print("\nObservation:")
print("- Noise increases false corner detections.")
print("- Higher noise produces more spurious corners.")
print("- Gaussian Blur reduces false corners and improves detection quality.")
```

[Choose Files](#) | input 4.jpg
Input 4.jpg(image/jpeg) - 45445 bytes, last modified: 10/8/2026 - 100% done
 Saving input 4.jpg to input 4 (4).jpg

Original
Corners = 355



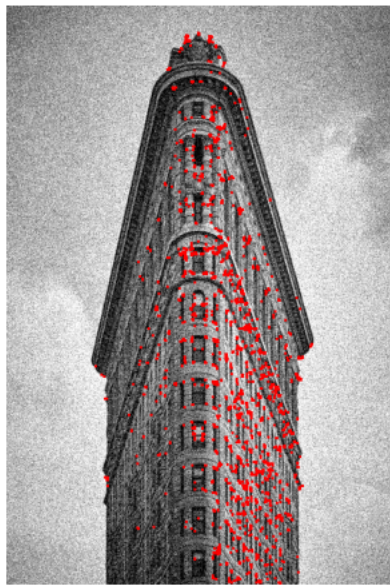
Low Noise
Corners = 368



Medium Noise
Corners = 382



High Noise
Corners = 494



Blurred + Harris
Corners = 312



=====
HARRIS CORNER DETECTION RESULTS
=====
Original Image : 355
Low Noise : 368
Medium Noise : 382
High Noise : 494
After Gaussian Blur : 312

Observation:
- Noise increases false corner detections.
- Higher noise produces more spurious corners.
- Gaussian Blur reduces false corners and improves detection quality.